Smart City Command Center - Project Report

This report describes what the project currently does, what is already implemented and verified, and what still remains if the goal is production deployment rather than a hackathon demo.

1. What the project is doing

- It turns an event input form into a traffic impact prediction workflow for Bengaluru.
- It trains a congestion model with XGBoost, using event type, crowd size, weather, time, location, and historical traffic.
- It returns a congestion score, risk level, peak time, affected roads, diversion routes, and police deployment guidance.
- It renders a command-center dashboard with seven screens: input, impact, heatmap, diversion, deployment, AI commander, and analytics.
- It logs predictions and model metrics into SQLite so the system can act like a learning loop.

2. What is already built and verified

Area	Current state
Frontend	Streamlit dashboard with dark command-center styling, maps, KPI cards, charts, and a demo preset button.
Backend	FastAPI service with /health, /train, and /predict endpoints (lifespan startup).
UI-backend link	Dashboard calls predict_event in-process by default; set SMART_CITY_API_URL to route through the FastAPI /predict endpoint, with automatic in-process fallback.
ML model	XGBoost congestion model trained and saved to disk.
Maps	Folium heatmap, hotspot markers, diversion routes, and deployment markers.
Analytics	Congestion, traffic volume, impact, resource allocation, and feature importance charts.
Persistence	SQLite tables for event predictions, AI messages, and metrics.
Tests	pytest suite (tests/) covering risk bands, prediction fields, resource scaling, API endpoints, validation, and DB logging - 9 passing.
Demo flow	Cricket Match / 50,000 crowd case returns about 91/100 congestion and a high-risk response plan.

3. Existing day-2 model results

Metric	Value
Clearance regressor	74.0 min MAE
Priority classifier	0.739 accuracy
Blackspots	184 clusters
Junction forecast	647 rows
Planned events	467 scored events
Surge detection	752 alerts

4. Smart City model metrics

Metric	Value
Model	XGBoost
Accuracy (binned LOW/MED/HIGH)	0.925
RMSE	4.335

R2	0.889
Data source	synthetic (build_training_frame)

Caveat: accuracy is derived by binning the regressor output into LOW/MEDIUM/HIGH, and all metrics reflect fit on synthetic training data. They must be revalidated against real labelled traffic before any operational or production claim is made.

5. Engineering fixes applied in this pass

A review verified the project runs and matches this report, then applied 9 fixes (all 9 complete and verified):

#	Fix	Status
1	Dashboard now routes predictions through the FastAPI /predict endpoint when SMART_CITY_API_URL is set (in-process fallback), so the UI is a real API client instead of bypassing the backend.	Done
2	Added missing httpx dependency (required by the FastAPI TestClient and the dashboard API client).	Done
3	Moved the 0.965 score fudge factor into config as a documented SCORE_CALIBRATION_FACTOR (presentation calibration, not a modelling correction).	Done
4	Replaced the deprecated FastAPI @app.on_event startup hook with a modern lifespan handler.	Done
5	Recorded a synthetic-data caveat in the metrics JSON and report so the scores are not mistaken for real-world skill.	Done
6	Labelled accuracy as binned LOW/MEDIUM/HIGH classification, separate from the RMSE/R2 regression metrics.	Done
7	Added a pytest suite (9 tests) for risk bands, prediction fields, resource scaling, API endpoints, input validation, and DB logging.	Done
8	Moved hard-coded peak-time and resource-plan magic numbers into config (PEAK windows, TIME_PRESSURE, RESOURCE_PLAN, RISK_BANDS).	Done
9	Pinned dependency versions to ranges verified on Python 3.14 for reproducible installs.	Done

6. What it still needs to do (production)

- Connect to real live traffic feeds instead of synthetic demo scenario data for the command-center layer.
- Replace the offline road snapshots with a real geospatial road network and live routing data.
- Add a true SUMO runtime integration for microscopic simulation beyond the fallback queue model.
- Retrain and revalidate the congestion model on real labelled traffic; the current metrics are on synthetic data.
- Add authentication, API rate limiting, and deployment monitoring before any public deployment.

7. Hackathon readiness summary

Assessment: the project is hackathon-ready. The demo flow works end-to-end, the UI is polished, the outputs are coherent for judges, and the 9 review fixes above are applied and verified. The main remaining gap is production realism (real data and validation), not demo completeness.

8. How to run

```
pip install -r requirements.txt
python run_all.py                       # Day 1-5 pipeline
pytest -q                               # run the test suite (9 tests)
uvicorn backend.api:app --reload        # start the API (optional)
```

```
# optional: route the dashboard through the API
#   set SMART_CITY_API_URL=http://127.0.0.1:8000
streamlit run app/dashboard.py
```